

Petrozavodsk 2026 Winter Day 1

KAIST Contest

Tutorial

Problem A. RUN Number

Problem Idea: Eunjae Jin (ai4youej)

Preparation: Eunjae Jin (ai4youej)

We aim to find the largest RUN number less than or equal to the current target and subtract it repeatedly, similar to a greedy algorithm. The specific strategy and its correctness are described below.

Algorithm

1. Define u_x as the repunit of length x (i.e., an x -digit number consisting of all 1s). For example, $u_4 = 1111$.
2. Given a k -digit number K , we select the largest integer S from the set

$$C = \{d \cdot u_{k-1} \mid 1 \leq d \leq 9\} \cup \{d \cdot u_k \mid 1 \leq d \leq 9\}$$

such that $S \leq K$. We add S to our solution set and repeat the process with $K - S$ until K becomes 0.

The intended solution operates in $\mathcal{O}(N^2)$ time complexity.

Proof of Correctness

Let u_k denote the repunit of length k . We define the proposition P_n as follows:

$$P_n : \forall x \leq u_{n+1}, x \text{ can be represented as the sum of at most } (n+1) \text{ RUN numbers.}$$

We prove that P_n is true for all $n \geq 1$ by mathematical induction. Since any N -digit number K satisfies $K < 10^N < u_{N+1}$, proving P_N implies that K can be represented as a sum of at most $N+1$ RUN numbers.

1. **Base Case (P_1):** Consider $x \leq u_2 = 11$. The integers $1, \dots, 9$ and 11 are essentially RUN numbers (sum of 1 term). The integer 10 can be represented as $9+1$ (sum of 2 terms). Thus, any $x \leq 11$ requires at most $1+1=2$ RUN numbers. P_1 holds.
2. **Inductive Step:** Assume P_{n-1} is true. We show that P_n follows. Let m be a positive integer such that $m \leq u_{n+1}$.
 - Case 1: $m = u_{n+1}$. This is a RUN number itself, so it requires 1 term. The statement holds.
 - Case 2: $m < u_{n+1}$. Since $u_{n+1} = 10 \cdot u_n + 1$, we have $m \leq u_{n+1} - 1 = 10 \cdot u_n$. We choose the largest integer $a \in \{0, 1, \dots, 9\}$ such that $a \cdot u_n \leq m$. Let $R = m - a \cdot u_n$. By the choice of a , it must be that $m < (a+1)u_n$, which implies:

$$R = m - a \cdot u_n < (a+1)u_n - a \cdot u_n = u_n$$

Since $R < u_n \leq u_n$, we can apply the inductive hypothesis P_{n-1} . Thus, R can be represented as the sum of at most n RUN numbers. Consequently, $m = a \cdot u_n + R$ can be represented as the sum of 1 (for $a \cdot u_n$) + n (for R) = $n+1$ RUN numbers.

Therefore, P_n is true. ■

Problem B. Two Histograms

Problem Idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

First, let's solve a simpler version of $K = 2$.

Let's simplify the condition for the 3rd grid. Since $(A \cup B)^C = A^C \cap B^C$, squares that are colored white should be a union of two white histograms, one falling from the top and one progressing from right to left. For every white cell, every cell right to it should be white, or every cell above it should be white. So no white cell has a black cell right to it and a black cell above it. Let's call this structure "L structure".

If we color a subset of cells of the third grid without making an L structure, we can color the rest without making an L structure. This can be done by coloring a cell white if and only if there is no black cell above it or right to it.

Now let's color the ends of $K \times 1$ rectangles. Let the WB rectangle be a rectangle where the left end cell is colored white and the right end cell is colored black. Let the BW rectangle be a rectangle where the left end cell is colored black and the right end cell is colored white.

For every $K \times 1$ rectangle, we can know whether there is a black cell right to the white cell regardless of how we color the other rectangles.

A white end cell has no black cell to its right if and only if the rectangle is of type BW and there is no rectangle to its right. Let a rectangle be a "special rectangle" if there is no rectangle to its right. Let other rectangles be non-special rectangles.

To avoid L-structures, the cell above the white end must not be black unless the rectangle is a 'special BW'. Let's call this condition as $\star$.

Consider the rectangles starting at a fixed x . Condition $\star$ implies that no BW rectangle can be placed above a WB rectangle. Thus, all BW rectangles must be placed below all WB rectangles. For p rectangles, there are only $p + 1$ valid configurations, determined by the count of BW rectangles.

Let $dp[i][j]$ be the maximum score achievable considering all rectangles starting at $x \leq i$, where exactly j rectangles starting at $x = i$ are colored BW.

The transition is:

$$dp[i+1][k] = \max_j (dp[i][j]) + score[i+1][k]$$

where the maximum is taken over all j compatible with k (satisfying condition $\star$). Here, $score[i+1][k]$ denotes the total score of rectangles starting at $x = i+1$, assuming the bottom k are colored BW and the rest are colored WB.

We verify the compatibility of having j BW rectangles at $x = i$ and k at $x = i+1$ in constant time. This yields a naive $O(N^2)$ DP solution. By some casework, dp transition could be done in linear time, resulting in $\mathcal{O}(N)$ solution.

Now, consider general K . The crucial observation is that a rectangle starting at x ends at $x+K-1$, interacting only with the rectangle starting at x or $x+(K-1)$. First, we identify which rectangles are special. Then, using the periodicity, we decompose the problem into independent chains based on $x \pmod{K-1}$, solving each as a separate $K = 2$ instance.

Problem C. One, Two, Three

Problem Idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

Let c be a sequence obtained from a by replacing every 3 with -1 . Let S denote the prefix sum array of c . Note that $c_i \equiv a_i \pmod{4}$ holds.

Lemma 1. *The entire sequence is removable if and only if $S_N \geq 0$ and S_N is divisible by 4.*

Proof. ($\rightarrow$) For every removed segment, the sum of c is divisible by 4 and is greater than or equal to 0.

($\leftarrow$) Let's use induction on N .

If $N \leq 4$, the sum of a is less than 12. Since $S_N \equiv 0 \pmod{4}$ and $S_N \geq 0$, the sum must be 4 or 8 (it cannot be 0 as $N \geq 1$). Thus, we can erase every element at once.

Now assume $N > 4$. First, we remove any "neutral" segments (segments with c -sum 0). If there are adjacent 3 and 1, or a group of one 2 and two 3s (like 233, 323, 332), we remove that segment. Since the sum of c for these segments is 0, the remaining sequence still satisfies the condition $S_{new} = S_N$, allowing us to apply the induction hypothesis.

The difference $X = S_q - S_p$ must be 4 or 8. (It must be positive since we removed 0-sum segments, and cannot be negative as that would imply many 3s, which contradicts the non-adjacency of 3 and 1 or 2).

If $X = 8$, since the length is at most 4 and $\max(c_i)=2$, the segment must be 2, 2, 2, 2. By choosing the first two elements (2, 2), we get a segment with sum 4. If $X = 4$, we simply take that segment.

In either case, we find a segment with c -sum 4. Removing this reduces S_N by 4. Since $S_N \geq 4$ (as $S_N > 0$ and $S_N \equiv 0 \pmod{4}$), the new total sum remains non-negative. By the induction hypothesis, we can remove the rest. $\square$

Let's define dp_i as a pair (minimum sum of a , $-$ maximum sum of b) for the prefix $1 \dots i$. We use lexicographical comparison for minimization. At step i , we have two choices:

1. Keep the i -th element: transition from dp_{i-1} .
2. Remove a segment ending at i : transition from some dp_j where the segment $(j+1, \dots, i)$ is valid.

The validity condition for removing segment $(j+1 \dots i)$ is that its c -sum is non-negative ($S_i - S_j \geq 0$) and divisible by 4 ($S_i \equiv S_j \pmod{4}$). The transition is:

$$dp_i = \min \left(dp_{i-1} + (a_i, -b_i), \min_{\substack{0 \leq j < i \\ S_j \leq S_i \\ S_j \equiv S_i \pmod{4}}} \{dp_j\} \right)$$

This gives a naive $\mathcal{O}(N^2)$ solution. To optimize, we need to efficiently query the minimum dp_j among those with $S_j \leq S_i$ having the same modulo 4. We can use 4 separate segment trees (one for each remainder) combined with coordinate compression on the values of S . This reduces the complexity to $\mathcal{O}(N \log N)$ for $p = 1$.

For the actual construction, we need to find the sequence of operations. The Lemma implies that if the total removal is possible, we can strictly find a local segment (sum of a being 4 or 8) to remove without violating the prefix sum condition.

We can implement this using a stack. We iterate through the sequence, pushing elements onto the stack. After each push, we check the top elements. If they form a removable segment (sum 4 or 8) and removing them does not cause the partial prefix sum of c to drop below zero (checking the condition locally), we pop them from the stack and record the operation. Since each element is pushed once and popped at most once, this approach works in $\mathcal{O}(N)$ time.

Problem D. Construct a Graph

Problem Idea: Yumin Park (flakepowders)

Preparation: Yumin Park (flakepowders)

Let's first determine the existence of such graph. If there exist i, j, k such that $D_{i,j} + D_{j,k} < D_{i,k}$, then the length of the path from i to k through j is less than $D_{i,k}$, so a graph satisfying the condition does not exist. Otherwise, a complete graph with the weight of the edge connecting i and j being $D_{i,j}$ satisfies the condition.

If an edge exists between two vertices u, v in the constructed graph, the weight of that edge must be $D_{u,v}$.

- If the weight is less than $D_{u,v}$, the shortest path length between u, v will become less than $D_{u,v}$.
- If the weight is greater than $D_{u,v}$, this edge will not affect any shortest paths. This is because a path with the shortest path between u, v instead of this edge will always be shorter. Since the goal is to minimize the total weight of the edges, there is no reason to include this edge.

Therefore, we can think of $D_{u,v}$ as the cost of connecting vertices u, v . Here's how to construct the graph: Iterate through the pairs of vertices (u, v) in ascending order of $D_{u,v}$. For each pair (u, v) , check if the length of the current shortest path between u and v is greater than $D_{u,v}$. If it is, you must directly connect the two vertices with an edge of weight $D_{u,v}$. This is because any other remaining vertex pairs that are not yet connected have a connection cost of $D_{u,v}$ or more, so they cannot affect the shortest path between u and v . After adding the edge, update the shortest paths that pass through this new edge in $\mathcal{O}(N^2)$ time. After iterating through all elements of D , verify if the shortest paths in the constructed graph are correct. If they are, return the graph. Otherwise, return that no graph satisfies the conditions.

In fact, there is no need to manage the shortest paths in the graph we're constructing. This is because when iterating a vertex pair (u, v) , the shortest path length between any pair of vertices (i, j) with $D_{i,j} < D_{u,v}$ will already be equal to $D_{i,j}$.

- This is obvious if i and j are directly connected.
- If i and j were not directly connected, it implies that the shortest path length between them was already $D_{i,j}$.

Also, if a path of length $D_{u,v}$ exists between u and v without directly connecting them, the weights of the edges forming the path will be strictly less than $D_{u,v}$. Therefore, if any vertex $m (m \neq u, v)$ is on such path, it must satisfy $D_{u,v} = D_{u,m} + D_{m,v}$. The existence of such vertex is equivalent to the existence of such path, and we can check it in $\mathcal{O}(N)$ time.

Problem E. Sequence and Queries

Problem Idea: Seyun Roh (knon0501)

Preparation: Seyun Roh (knon0501)

Let's define $g(i, j)$ as follows:

$$g(i, j) = \sum_{k=1}^{\min(n-i+1, n-j+1)} f(i, j, k)$$

Then, the following equality holds for $1 \leq i, j \leq n$:

$$g(i, j) = \begin{cases} g(i+1, j+1) + 1 & \text{if } s_i \leq s_j \\ 0 & \text{otherwise} \end{cases}$$

and $g(i, n+1) = g(n+1, j) = g(n+1, n+1) = 0$.

Therefore, the sum of all $g(i, j)$ can be calculated in $\mathcal{O}(n^2)$ by using dynamic programming.

Problem F. Monochrome Tree

Problem idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

Let's denote a good pair as a black vertex pair where one is an ancestor of another.

Lemma 1. *If there is vertex x which is colored white and its parent is colored black, exchanging color makes the score equal or smaller.*

Proof. Denote parent of vertex x as y . Let's count the good pairs where one of the vertices is x or y .

Number of good pairs where x or y is a descendant does not change.

Let z be a black vertex. For z to be a descendent of x , z should be in a subtree of x . For z to be a descendent of y , z should be in a subtree of y . Since subtree of y includes subtree of x , number of good pairs where x is an ancestor is smaller or equal than the number of good pairs where y is an ancestor.

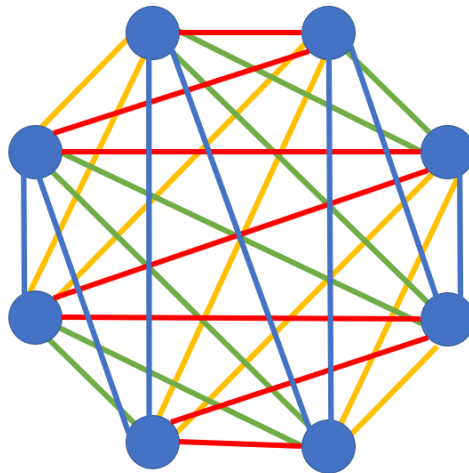
□

Now we know that there exists an optimal solution where for every black vertex, every descendent is a black vertex. Let's count number of good pairs by fixing the ancestor. The number of good pairs will be sum of (subtree size - 1) for every black vertex. The answer is $c_k = (\text{sum of smallest } k \text{ subtree size} - k)$.

Problem G. Clique partition

Problem idea: Junhyuk Song (SongC)

Preparation: Junhyuk Song (SongC)



If N is an even number, we can decompose the clique to trees with out any duplication of edges. We can even split it into paths instead of general trees. The diagram at the top shows the way of decomposition. In odd case, just solve for $N - 1$ and add a star graph around the node N .

Problem H. Good night

Problem idea: Junhyuk Song (SongC)

Preparation: Junhyuk Song (SongC)

$$N \leq 100$$

To construct a naive polynomial algorithm, we should observe some properties of streetlights.

- If a streetlight goes out and doesn't turn back on immediately, it will remain off permanently.
- Over time, the area of illumination along the line can only decrease.
- If any streetlights don't go out for a time interval of T , then that state will be maintained permanently.

The proofs have been left out, to be completed as an exercise by the reader. By these facts, we can construct an $O(N^3)$ simulation algorithm. For each period, examine every light to determine if it turns back on immediately after going out. Testing a single light requires $O(N)$ time, so the time complexity for each period is $O(N^2)$. Based on the third property, at least one light will go out during each period, except for the last one. So there are at most N periods and the overall complexity of simulation is $O(N^3)$.

$$N \leq 1000$$

We can further optimize the simulation algorithm from subtask 1. By using a segment tree, we can efficiently maintain the state of the line. Specifically, the tree will store the number of lights illuminating each segment of the line. To check whether a position is lighted by streetlight at a specific time, we can perform queries on the segment tree. If the number of lights that go out at that time is denoted as m , then the number of required queries is $O(m)$. Notably, the cumulative number of lights going out in a period is $O(N)$. As a result, the complexity of the simulation is improved to $O(N^2 \log N)$.

All A_i are distinct

By the constraint, there are no two lights going out at the same time. In our simulation, checking each light individually is inefficient. Instead, we should focus on the lights which are remained off permanently when it goes out. If we have the set of such lights denoted as S , the following method works correctly:

1. Pop the light that goes out earliest from S .
2. Remove it and update the line's state.
3. Update S by adding lights which newly satisfy condition due to elimination of a light.
4. Repeat 1-3 until S is empty.

Notice that we don't have to delete any light on update at (3) since the area of illumination can only decrease. If we can maintain S with priority queue and find new lights added to S fast enough, the whole problem can be solved efficiently.

To detect new lights, consider the condition for x to be in S . It is equivalent to the following: There exists a point lying on the path between origin and x which is either not illuminated by any light or is solely illuminated by x .

Now we can optimize the algorithm with a segment tree. Construct a minimum segment tree with lazy propagation. Begin by recording the number of lights illuminating each segment, then determine the initial S by just checking for every lights. Proceed as mentioned:

1. Remove a light from S , then decrement the corresponding segment in the tree using lazy propagation.
2. Check the tree's minimum value. If it's zero, a point remains unilluminated. All lights behind this point can be added to S . (To optimize, pinpoint the leftmost zero or ignore previously detected points. Ignoring can be done by assigning a sufficiently large value to the point.)
3. Similarly, detect points illuminated by just one light and assess them. To ascertain the exact index of a light, construct another tree storing the sum of light indices. With only one light illuminating a point, its identification is possible through a point query on this secondary tree.

Since the light count for a specific point decreases, every unit segments are detected at most twice. The algorithm requires $O(N)$ segment tree queries and the overall time complexity is $O(N \log N)$.

Full solution

$O(N \log^2 N)$ Solution

Let's try to apply the solution of subtask 3. The only difference between subtask 3 and the full task is existence of multiple lights which go out simultaneously. Most parts of the algorithm are reusable, but we have to revise the condition to be included in S . For the full task, let's try to modify the condition like the followings.

- There exists a point lying on the path between origin and x which is not illuminated by any light.
- There exists a point lying on the path between origin and x which is only illuminated by lights whose value of A is same with x .

This isn't a strict equivalence, as there might be a light x meeting the second criterion but could potentially be turned on later. Think of scenarios where a light, sharing the same A value as x , re-lights post-extinguish, lighting the route to x . Pinpointing a strict equivalent condition for S is challenging. However, if we can identify at least one of the fastest extinguishing lights from S , the problem becomes solvable. (Consider Dijkstra algorithm)

Modify the second criterion to: There exists a point on the path between the origin and x which is illuminated only by lights sharing x 's A value and are **behind** x .

Upon any light extinguishing, at least one light satisfies this modified condition. Focus on the leftmost light extinguishing simultaneously.

The problem can now be solved using a complex data structure. Most parts of the algorithm mirror subtask 3. After each deactivation, we now need to pinpoint lights meeting the modified conditions. When focusing a line's unit segment, consider events triggering updates to the PQ:

1. The number of distinct A values illuminating the segment becomes one.
 2. The segment, already lit by a singular A value, sees a shift in the leftmost illuminating light's position.
 3. All illuminating lights are removed.
-

For a segment tree storing lights, each node should contain a set of pairs representing lights illuminating the entire segment in the form (A_i, X_i) . During initialization, assign pairs to nodes for all lights, with each light setting $O(\log N)$ pairs.

For light elimination, tree maintenance is simplified by erasing $O(\log N)$ pairs from sets. Our focus now shifts to updates. Nodes with more than two distinct A_i values will always remain lit, regardless of the number of lights extinguishing simultaneously. Thus, such nodes need no attention. An update for a leaf node demands $O(\log N)$ within our structure. (Just an application of a point query on a segment tree)

For case 1 and 3, they happen only one time for a node. So, just use point update for all leaves under it. Then total number of update is $O(N \log N)$ and total time complexity is $O(N \log^2 N)$.

For case 2, we will focus on each node in the same way to case 1. Denote the minimum of X_i for a node as v . As time progresses, v ascends and v affects to lights which are further left than v . So if v becomes to v_1 to v_0 , we can just update for each nodes in $(v_0, v_1]$. The whole complexity about this case is bounded to $O(N \log^2 N)$.

Finally, we can implement all operations we need in amortized $O(N \log^2 N)$.

$O(N \log N)$ Solution by Sjimed

Let's think about that the point x is whether reachable or not, at the time t . We may assume light $1, 2, \dots, n$ is not turned off right before t and $A_1 \equiv A_2 \equiv \dots \equiv A_k \equiv t \pmod{T}$ ($1 \leq k \leq n$). We just want to compute the minimum $x > 0$ such that x is not reachable at t .

In this perspective, the light $i \leq k$ indicates that if we can reach $\max(L_i, X_i)$, we also can reach R_i . For the lights that $i > k$, we can reach R_i if we can reach L_i . Therefore, the problem could be changed to finding the minimum x which is not covered by the intervals: $[\max(L_i, X_i), R_i]$ for $1 \leq i \leq k$, and $[L_i, R_i]$ for $k < i \leq n$. So basically the light i covers $[L_i, R_i]$, but $[\max(L_i, X_i), R_i]$ if $A_i \equiv t \pmod{T}$ (the interval $[L_i, \max(L_i, X_i))$ is erased). If currently there is only one $0 \leq t < T$ such that $\exists i, A_i = t$ and $x \in [L_i, \max(L_i, X_i))$, x would be light off when the time $\pmod{T}$ becomes A_i .

Therefore, if we could manage the number of times that cover the point, we could manage the set S .

Currently, our programming task is:

1. Count the number of the times that $[L_i, \max(L_i, X_i))$ covers the point and find the minimum
2. If there exists x that the count value is at most 1 and there is no turned-on light that $x \in [\max(L_i, X_i), R_i]$, update x in S and erase x .

By using the minimum segment tree with lazy propagation, we can get the point that the count value is less or equal to 1. We also can eliminate all $[\max(L_i, X_i), R_i]$'s of turned-on lights and delete a point after putting into S simply adding 2 (this makes all the points in the interval never come out).

Since all lights are turned off according to the order of X_i among the lights having the same A_i 's, we can pre-compute the changing intervals using data structures such as std set. All lights are turned off at most once, so the total update query of the segment tree is $O(N)$.

Finally, we just manage the queries some light is actually turned off or some point is not reachable anymore, using a priority queue.

Total time complexity of the solution is $O(N \log N)$

Problem I. Remix

Problem idea: Issa Hazem

Preparation: Issa Hazem

Notice how any new number we make can only be smaller than or equal to $\max(t)$, so the optimal answer can't be larger than the $\max(s)$. Let e be the subset we do the final operation on, then the final number we end up with is equal to $\max(e) - \min(e)$. We can make this value greater by either increasing the value of $\max(e)$ or decreasing the value of $\min(e)$. But as we have seen any operation done on a subset may only decrease its max value, so the max value, optimally, will be a number from the original set. So our only option is to try and decrease the value of $\min(e)$.

Now we just need to solve the following problem: we have a multiset s , and we can choose any subset t and replace it with one element equal to $\max(t) - \min(t)$. Minimize the minimum element in the final set. Note that we don't have to end up with one integer like in the original problem and therefore we may ignore the scheme of choosing a subset altogether and only focus on choosing the two integral values, $\min(t)$ and $\max(t)$, a subset of size two if you must.

Let's think of all the non-empty subsets of s , and let c_i be the sum of elements of the i -th subset. The lower bound for the value of $\min(e)$ is the lowest value for $|c_i - c_j|$ where $i \neq j$. And it turns out that there is a construction to achieve this value. Let A and B be the i -th and j -th subsets for which $|c_i - c_j|$ is minimum, if there's any element that occurs in both sets remove it from both and it will not affect $|c_i - c_j|$. Now do the following until one of the sets becomes empty:

1. Choose any element x from A , and choose any element y from B .
2. Do an operation with the subset $\{x, y\}$.
3. Erase x and y from A and B respectively.
4. Insert $x - y$ to A if x is greater than y , and insert $y - x$ to B otherwise.

Notice how $|\sum A - \sum B|$ is equal to $|c_i - c_j|$ after each operation. In the end, we will be left with exactly one integer with the value $|c_i - c_j|$ in one of the sets. Note that we must be left with exactly one integer, not two or more because otherwise, it would contradict that $|c_i - c_j|$ is minimum as we could have removed the extra element(s) and achieved a smaller difference.

To find c_i and c_j , we can use knapsack dp but this only works for sufficiently small values of n and s_i . To achieve a better result, we need to notice that the absolute minimum value for $|c_i - c_j|$ is 0 which only happens when $c_i = c_j$. Since the values in the multiset are bounded by some maximum value, let's say M , we can show that any multiset of size larger than some value C must contain two disjoint subsets with equal sum. In a multiset of size C , there are a total of 2^C subsets and the maximum sum for any subset is CM . As a result, if $2^C > CM$, there must exist two disjoint subsets having equal sum by the pigeonhole principle. So we just need to choose the minimum C satisfying the mentioned equation then compute all the subset sums of any C elements in $O(2^C)$.

Time Complexity: $O(n + 2^C)$ where C is the minimum multiset size such that there exist two subsets with equal sum ($C \leq 22$ for $s_i \leq 10^5$).

Problem J. Binary sequence and queries

Problem idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

If $x = 0$ and $y = 0$, there is no such segment. Let's think when $x \neq 0$ or $y \neq 0$. Define each $f(s, e), g(s, e)$ be the longest consecutive 0s' and 1s' of segment $[s, e]$.

Theorem 1. *A subsegment $[s, e] \subseteq [l, r]$ satisfying the conditions exists if and only if $x \leq f(l, r)$ and $y \leq g(l, r)$.*

Proof. ($\leftarrow$) Since any subsegment cannot have longer consecutive runs than the original segment, the condition is necessary.

($\Rightarrow$) We construct the segment $[s, e]$ in two steps using minimality.

Let k_1 be the minimal index such that $f(l, k_1) \geq x$ and $g(l, k_1) \geq y$. Since k_1 is minimal, adding a_{k_1} must have satisfied at least one condition exactly. Without loss of generality, assume $f(l, k_1) = x$ and $g(l, k_1) \geq y$. This implies that the suffix of $a[l \dots k_1]$ is exactly x zeros (denoted as block B_0).

Now, let k_2 be the minimal index in $[l, k_1]$ such that $f(k_2, k_1) \geq x$ and $g(k_2, k_1) \geq y$. We claim that for this segment, $f(k_2, k_1) = x$ and $g(k_2, k_1) = y$.

By definition, $f(k_2, k_1) \geq x$. Also, since $[k_2, k_1] \subseteq [l, k_1]$, by monotonicity, $f(k_2, k_1) \leq f(l, k_1) = x$. Therefore, $f(k_2, k_1) = x$.

Assume for the sake of contradiction that $g(k_2, k_1) > y$. Since $g(k_2, k_1) > y \geq 0$, the segment $[k_2, k_1]$ contains at least one '1'. However, we know the segment ends with block B_0 (which contains x zeros). Since there is a '1' in the segment, the start index k_2 cannot be inside B_0 . Thus, removing a_{k_2} does not affect the block B_0 , so $f(k_2 + 1, k_1) = x$. Also, removing a single element reduces g by at most 1. So $g(k_2 + 1, k_1) \geq g(k_2, k_1) - 1 \geq y$. This means the segment $[k_2 + 1, k_1]$ also satisfies both conditions. This contradicts the minimality of k_2 . Thus, we must have $g(k_2, k_1) = y$.

We have found a segment $[k_2, k_1]$ with exactly $f = x$ and $g = y$. □

$f(l, r)$ and $g(l, r)$ can be solved in $O(r - l)$, to decide if a segment that satisfies the given condition exists. If there exists such segment, we can find k_1 and k_2 by sweeping k_1 from l to r , then sweeping k_2 from k_1 to l . This can be done in $O(r - l)$. Since every query can be solved in $O(N)$, the whole problem can be solved in $O(QN)$.

For the full solution, let's define a segment tree where node for $[l, r]$ stores the following information.

- $f(l, r), g(l, r)$ (max consecutive 0s / 1s)
- $pref_0, pref_1$: length of consecutive 0s / 1s starting from l
- $suff_0, suff_1$: length of consecutive 0s / 1s ending at r
- length of the interval

Since we can merge two adjacent nodes, we can make a segment tree. Making a segment tree takes $O(N \log N)$ time. For each query, solving $f(l, r)$ and $g(l, r)$ costs $O(\log N)$ time. We can solve k_1 and k_2 in subtask 2 by using binary search. Since each value requires $O(\log N)$ time, we can solve k_1 and k_2 in $O(\log^2 N)$ time.

Overall we can solve the problem in $O(N \log N + Q \log^2 N)$ time. We can further optimize finding k_1 (and similarly k_2) by using Segment Tree Descent (walking down the tree). This eliminates the binary search overhead, reducing the time complexity to $O(N \log N + Q \log N)$

Problem K. Merging Branches

Problem Idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

Lemma 1. *Let $Len(k) = r_k - l_k$ be the length of the k -th branch. The minimum cost K to merge branches from s to e is:*

$$K = \max_{s \leq i < j \leq e} \left\lceil \frac{\max(0, l_j - r_i - \sum_{k=i+1}^{j-1} Len(k))}{j - i + 1} \right\rceil$$

(Interpretation: The total "gap" distance between branch i and j must be covered by the expansion of $j - i + 1$ branches.)

Proof. Necessity ($K \geq \mathbf{RHS}$): Consider any sub-segment of branches from i to j . The total distance from the end of i (r_i) to the start of j (l_j) is $l_j - r_i$. The sum of lengths of intermediate branches is $\sum Len(k)$. The remaining distance must be covered by the expansion of all $j - i + 1$ branches involved. Each branch contributes at most K . Thus, $(j - i + 1)K \geq \text{Gap Sum}$, which implies the formula.

Sufficiency ($K \leq \mathbf{RHS}$): We prove that if K satisfies the condition for all pairs, the greedy strategy works. Execute the greedy algorithm: iterate from $i = s$ to e , shifting/expanding each branch as far left as possible (up to connection) and then extending right. Suppose the component breaks between some branch m and $m + 1$. This implies the total reach of branches $s \dots m$ was insufficient to bridge the gap to $m + 1$. This failure can be traced back to a specific range $[i, j]$ where the "density of gaps" exceeded the available expansion budget $(j - i + 1)K$, contradicting our assumption.

□

Full Solution: We can precompute the values using Dynamic Programming. Let $DP[i][j]$ be the answer for the query (i, j) .

$$DP[i][j] = \max(DP[i][j - 1], DP[i + 1][j], \text{NeededCost}(i, j))$$

where $\text{NeededCost}(i, j)$ is the value calculated by the Lemma for the pair (i, j) specifically. Since $N \leq 5000$, we can compute this table in $O(N^2)$. Then, each query (s, e) is simply $DP[s][e]$, answered in $O(1)$.

Problem L. Simple Game

Problem idea: Geunyoung Jang (azberjibiou)

Preparation: Geunyoung Jang (azberjibiou)

Assume $\gcd(a, b) \neq 1$. Every element in the sequence can be expressed as $a + bx$ where x is an integer between 0 and $2n - 1$. Since $\gcd(a, b)$ divides a and b , $\gcd(a, b)$ divides $a + bx$, which means that $\gcd(a, b)$ divides every element in the sequence. So you cannot proceed the operation.

Now let's think when $\gcd(a, b) = 1$. By euclidean algorithm, $\gcd(a + ib, a + (i + 1)b) = \gcd(a + ib, b) = \gcd(a, b) = 1$. This means that any two adjacent elements in the sequence are coprime. For i th operation, you can choose $a + 2ib$ and $a + (2i + 1)b$.

Problem M. Tsunami

Problem idea: Hongyoon Mun (mhy908)

Preparation: Hongyoon Mun (mhy908)

Let's construct a DP table. Let $DP[i][j]$ be minimum time to arrive at point (i, j) .

Then $DP[i][j] = \min_{0 \leq i' \leq X+1} (DP[i'][j-1] + \text{cross}(i', j) + c_{j-1} \times |i' - i|)$, where $\text{cross}(i', j)$ is the time crossing an obstacle placed at (i', j) .

If (i, j) is an evacuation spot, we should also consider the r value of that point.

By naively calculating the above DP, we get $O(X^2Y)$.

Note that most of the DP values don't change as i increases. It only changes when there is an event at that particular coordinate.

When there is an evacuation spot at (i, j) which has value r , we can consider $r + c_j \times |i - i'|$ for a new candidate of $DP[i'][j+1]$. If there is an obstacle, we just use the original equation. With this optimization, we get $O(XY + NX + MX^2)$.

We now need to make further optimizations for the obstacle case.

The reason why it takes $O(X^2)$ time is because we tried all possible starting x-coords and finishing x-coords for moving horizontally. Instead, we can just traverse the 1 dimensional array $DP[*][j]$ for fixed j and maintain prefix/suffix minimums of $DP[i][j] - c_j \times i / DP[i][j] + c_j \times i$ respectively. With these values, we can consider horizontal moving at obstacle coordinates achieving $O((Y + N + M)X)$ time complexity.

Another way to solve in this time complexity is observing that there is always an answer that only move horizontally at evacuation points or the endpoints of obstacles. We can prove this easily with the fact that c is a nondecreasing sequence.

Therefore, we can just think that there is a new evacuation point at endpoints (endpoints will be $s-1, e+1$ if the obstacle is ranged at $[s, e]$) of obstacles having r value of $DP[s-1][j]$ and $DP[e+1][j]$, and modify the original DP equation to $DP[i][j] = DP[i][j-1] + \text{cross}(i, j-1)$.

For the full task, we need a data structure to handle range addition, range line insertion and maintain the minimum y-values for all x-values.

There are 2 possible ways to construct this structure.

A straight-forward way is to modify a li-chao tree to handle those instructions. Range line insertion is easy, just find $O(\log N)$ appropriate subtrees and do original line insertions on those subtrees. Time complexity will be $O(\log^2 N)$.

Range addition can be implemented with lazy propagation, but additional updates need to be made. We define a "flush" mechanism as updating a line written in a node to both to the left subtree and right subtree, and filling the original node with infinity. This will take $O(\log N)$ time, but flushing does not change the minimum values for all x's.

Therefore if we combine flushing and lazy propagation, we can solve the full problem in $O((N + M)\log^2 X)$. Further explanation is written at <https://codeforces.com/blog/entry/86731>.

Another approach needs additional observation that when we update a line, no more than one continuous range gets covered by that line. One can prove it using that c is a nondecreasing sequence. Therefore if we already know each x-coordinates' minimum y values, we can apply binary search to find the appropriate

range, and cover the range by the given linear function. Now, this can be implemented by lazy segment tree in $O((N + M)\log^2 X)$. This approach can be improved by taking out the $\log X$ term in binary search using internal binary search in the segment tree leading to $O((N + M)\log X)$, but not required to solve the full task.

Problem N. Discount Event

Problem Idea: Youngwoo Park (flappybird)

Preparation: Youngwoo Park (flappybird)

Let's simplify the problem. The given cities and roads can be thought of as a weighted tree. For each query, we need to find the diameter when the weight of the path ending at a given vertex in the tree becomes 0. Let's view the path with 0 weights as a single "compressed node". The new diameter is either:

- Entirely contained within one of the hanging subtrees connected to the path.
- Formed by connecting the deepest paths from two different hanging subtrees via the zero-weight path.

For simplicity, let's root the tree. Let T_v be a subtree of v , and p_v be the parent of v .

Let's precalculate the following values for every vertex v . It can be done by simple dynamic programming.

- Maximum length of path from p_v in $T_{p_v} \setminus T_v$
- At most 3 maximum length of path from v in T_v
- Maximum length of path from v in $(T_{root} \setminus T_v) \cup \{v\}$
- Diameter of T_v
- Diameter of $T_{p_v} \setminus T_v$
- Diameter of $T_{root} \setminus T_v$

For a query (u, v) with $l = \text{LCA}(u, v)$, the path is decomposed into two vertical segments: $u \rightarrow \dots \rightarrow \text{child}_u(l)$ and $v \rightarrow \dots \rightarrow \text{child}_v(l)$. We use Binary Lifting (Sparse Table on Tree). $\text{Table}[u][k]$ stores the aggregated (Max Diameter, Max Depths) for the path segment of length 2^k above u . Using this table, we can aggregate info for the two vertical segments in $O(\log N)$. Finally, at the LCA node l , we merge:

- Aggregated info from the vertical segments.
- Info from the "Upper Tree" (parent side of l).
- Info from other children of l (excluding the two on the path).

The answer is $\max(\text{Max Diameter found}, \text{Sum of two largest Depths found})$.
